Before the First Mark

Jeremy H. Carroll

2026-10-09

Version 1.0.0

Abstract

Before interpreting <>, distinguish receiving an inscription from admitting an object and distinguish an external reader's resources from a regime's operations. This paper combines the source-grounded Pre-Genesis experiments with the Genesis account of finite admission, observation and response selection. The experimental record contains three runs, with no recorded model identifier or generation settings; it supplies examples of failure, not performance estimates. The mathematical account gives exact criteria for recovering answers from reports, a two-item separation minimum, classification of its ties, and a finite closure bound. It also records the closed Genesis continuation study and its proofs of admission growth and no return, while preserving the source's unresolved completion obligations. These results support precise questions about presentations and question-relative report reconstruction. A separately reported Rust construction connects primitive admissions with finite incidence complexes and exact rational homology. The standalone replay in this package covers the primitive layer; the construction's execution correspondence and chosen mathematical interpretation have separate scopes.

1. Starting before <>

The first question is what has been supplied and what has been licensed to happen with it. The author, reader and computer already use a metalanguage. This paper makes that fact explicit; it does not describe an observer-free creation from nothing.

Condition	Host-level description	What has not followed
Absent input	No payload was supplied, for example None in a declared option type.	No object-level empty word or ground has been produced.
Empty word	The word epsilon in a declared free monoid has length zero.	It need not be accepted by a selected grammar start.
Literal <>	Two ASCII characters, possibly one opaque token under a stated lexer.	Neither an empty set nor a singleton carrier is forced.
Admitted ground	A named initial object under an explicit regime definition.	Its carrier, operations and permitted transitions still need definitions.

"Before" here marks the order of questions in the account. It is not a derivation of physical time. Even writing None presupposes a host capable of representing absence. Likewise, a grammar production cannot demonstrate that there is no grammar in the metalanguage in which that production was written.

Let a declared transition system have state set X and transition relation T. If T is empty, no state has a T-successor. This follows from the definition of an empty relation. It does not say that an

external caller cannot provide more data, or that every later rule is prohibited. External supply and admission are events whose effects must be specified.

2. What the Pre-Genesis trials actually recorded

The Pre-Genesis strand contains the experiment design, Trial 1, Trial 2 and a full-history replay. Its trial record points to the original transcripts. The prompts withheld the intended comparison theories and asked the model to distinguish external presentation from internal expressibility. "No move is forced" was an admissible outcome. The next inscription was not prescribed as a successful rediscovery of an existing calculus.

The record supports the following limited observations:

Event	Recorded behavior	Evidential boundary
Bare marks and punctuation controls	A fenced prompt elicited familiar interpretations; raw controls elicited different replies.	The runs did not isolate every causal difference between conditions.
No declared continuation rule	All three runs said that a successor was not forced; some proposed a continuation under an added assumption.	A proposal and its adoption are different events.
`<a>` supplied after `<>`	Replies differed between presentation-relative and inscription-relative readings of identity.	The identity convention had not been fixed.
A separation task about `<a>` and `<ab>`	A reply treated observational indistinguishability as identification; another used a stale account of the current state.	Lack of a distinguishing operation does not itself install an object-language equality rule.
Truncated audit versus replay	An auditor examining only the local response missed a historical-state mismatch.	Local consistency alone did not establish consistency with the adopted history.

Trial 1 also bundled steps and fenced controls that the design had intended to send separately and raw. These are recorded design deviations. The model identifier, settings and dates were not recorded. There were no host-checker verdicts on the replies and no live PFP evaluation. The earlier invented "50 trials" table is not part of the evidence used here. No rate, benchmark advantage or theory-rediscovery claim follows from this record.

The failure-mode chapter names semantic overcompletion, premature relation, false provenance, lost state, stale certification and bundled extension. These are retrospective descriptions. Naming a failure does not implement a detector for it.

3. Six operations that must remain distinct

The trials become easier to interpret when the following operations have separate signatures and records:

Operation	Meaning in a declared host
Mention	Refer to an inscription or possible object in a discussion.
Supply	Deliver a payload to an interface.
Designate	Identify which presentation is currently under consideration.
Admit	Add a datum, form, instance or rule according to an admission policy.
Retain	Preserve an admitted object or event for subsequent use.
Operate	Apply an admitted transformation with stated preconditions.

A response proposing `<a>` does not establish that `<a>` was adopted. A later task that mentions `<ab>` need not erase `<a>`. A checker cannot reconstruct these facts from punctuation alone; it needs event semantics and enough of the event record.

The trial method is consequently modest: keep discovery prompts free of expected solutions, preserve the exact controls, separate steps, record adoption explicitly, and bind an audit to its actual context. These are design lessons from the record, not experimentally established universal remedies.

4. Observation-relative difference

Let A be a declared carrier and O a family of functions on A. Write

$$x \sim_O y \quad \Longleftrightarrow \quad o(x) = o(y) \text{ for every } o \in O.$$

Equality here is metatheoretic equality in each observation's codomain. If O is empty, every pair is equivalent. This does not imply that the inscription strings are equal or that the regime contains an equality operator.

Proposition 1 — separation cannot be recovered from a collapsed quotient. Suppose x and y have the same O-profile, while a task f requires f(x) different from f(y). No function on the current observational quotient can answer f on all inputs.

Proof. The quotient map q sends x and y to the same class. If f=g composed with q, then f(x)=g(q(x))=g(q(y))=f(y), a contradiction. Therefore a representation supporting this task must preserve more information than q preserves. This may be achieved by adding an observation that separates the pair. The proposition does not select that observation. □

For `<a>` and `<ab>`, a predicate for presence of b and a predicate for payload length greater than one both separate the pair, under a supplied string decoder. They disagree on `<aa>`. Full inscription identity also separates the pair and retains additional information. These are alternative admitted solutions, not capabilities obtained without a decoder or comparison operation.

5. A history projection can destroy the answer

Let H be a set of admissible histories, $\sigma : H \to S$ the state or protected answer required by a task, and $\pi : H \to V$ the report shown to an auditor.

Theorem 2 — history report criterion. An answer map $A : \pi(H) \to S$ satisfying $A(\pi(h)) = \sigma(h)$ for every $h \in H$ exists if and only if

$$\pi(h) = \pi(h') \quad \Longrightarrow \quad \sigma(h) = \sigma(h').$$

When it exists, it is unique on the report image. In particular, two histories with the same report and different required answers rule out every universally correct auditor seeing only that report.

Proof. A factorization gives $\sigma(h) = A(\pi(h)) = A(\pi(h')) = \sigma(h')$. Conversely, define $A(\pi(h)) = \sigma(h)$; the displayed condition makes this well-defined. Every member of $\pi(H)$ has this form, so the factor is unique. For a finite listed history population with decidable report equality, the construction is executable: find the first matching report and return that history's answer. Extending the map outside $\pi(H)$ requires a chosen answer there; no such extension is needed for reconstruction on admissible inputs. □

The source establishes the finite natural-number version as Compression.factors_iff_fiberConstant and factor_unique_on_image (source). The carrier-general statement and uniqueness on the report image are mechanized separately as GenesisPaper.history_report_criterion and history_factor_unique in the bundled lean4/Paper.lean. The existence direction uses Classical.choice; the finite source specialization is constructive.

This is an information-loss theorem, not a theorem that every task requires every raw token of history. A sufficient state summary may preserve σ. Conversely, more context alone does not guarantee that an auditor will use it correctly. Raw history retention is a useful provenance policy; task-sufficient reconstruction is the mathematical requirement.

The local response in the trials could be consistent with its asserted premise and still be inappropriate for the adopted state. Checking internal steps and checking the binding of their premises to the actual history are separate tasks. The report criterion makes this distinction precise for any specified answer function on admissible histories.

6. Genesis as an explicit finite admission study

The Genesis education account tracks the external 19L57G1 study. Here the source is pinned to f1d56c010e6ba10a3b9418bd7cc5953c067250f2. The earlier sealed tag sealed-derivation-v1 (3100c4e) and September 25 review (b368f28c) retain their own scopes. The education implementation covers a selected family; it is not a proved copy of every source research branch.

A finite regime is

$$R = (D, F, I, U, Q).$$

The admitted data have indices $D = \{0, \ldots, n-1\}$, with opaque presentation labels retained in the token-aware model. The indexed form list F assigns a positive arity a_f to each form. The set I contains instances $(f, (d_1, \ldots, d_{a_f}))$ over admitted data; U contains forms carrying a uniform all-tuples rule; and Q contains supplied requirements (i, j) with $0 \le i < j < n$. Each requirement is an unordered separation obligation $\{i, j\}$, stored as $(\min(i, j), \max(i, j))$; the stored order is not an orientation. These are the declared host structures through which the primitive operations are specified.

Regime isomorphisms are bijections of datum and form indices preserving form arities, ordered instance arguments, uniform-rule membership and these unordered requirements. Mapped requirement endpoints are sorted before comparison. Opaque presentation labels are erased before this comparison and do not constrain the bijections. Thus exchanging the endpoints of a requirement can identify two responses; an equivalence relation preserving directed requirements would be a different convention.

Event family	Declared effect and admission condition
Supply	Leave the admitted regime unchanged; the event may still be retained in the history.
AdmitDatum	Append an opaque label unless it is already admitted; repeated label admission designates the existing datum.
AdmitForm	Append a new form with positive arity, even if another form has the same arity.
AdmitInst	Insert an instance only when the form, arity and datum indices are valid.
AdmitRuleAll	Insert a uniform rule only for an admitted form.
Impose separation	Insert the canonical pair (i, j) for an unordered requirement only when $0 \le i < j < n$.

Instances, rules and requirements are retained as sets. These event semantics are defined in the source model; ill-formed admissions are refused. Reading L starts with <> supplied but unadmitted. Reading F also admits that initial token. Both use the same deduplicating admission convention thereafter. Neither reading is selected by inspecting the glyphs.

Let $\mathrm{Ctx}(R)$ contain exactly the patterns whose form is admitted, whose slot count equals its arity, whose fixed slots name admitted data, and which contain at least one hole. Plugging d replaces every hole by that same datum. Then

$$R \vdash (f, \mathbf{d}) \iff (f, \mathbf{d}) \in I \text{ or } f \in U, \qquad \mathrm{Prof}_R(d) = \{C \in \mathrm{Ctx}(R) : R \vdash C[d]\}.$$

Derivability here is evaluated on well-formed instances. Equal profiles define observational collapse; a context in their symmetric difference witnesses separation. With no admitted instances, every datum has the same profile, even if uniform rules are present: each such rule treats every plugging alike (collapsed_of_no_insts, source). Retaining another datum alone therefore need not increase expressive power.

The finite context grammar gives the exact count

$$|\,\mathrm{Ctx}(R)| = \sum_{f \in \mathrm{dom}(F)} ((n+1)^{a_f} - n^{a_f}).$$

For each form there are $n+1$ choices per slot, including the hole; subtract the n^{a_f} patterns without holes. The companion theorem GenesisPaper.contextWords_length formalizes the corresponding recursive enumeration count. contextWords_mem characterizes exactly the well-formed words containing a hole, and contextWords_nodup establishes that the enumeration has no repetitions.

Form identifiers keep distinct forms disjoint even when their arities coincide. These companion proofs are separate from the frozen source. Its explicit grammar partitions words into hole-free words and words containing a hole. Python and Rust independently enumerate the declared contexts, including repeated fixed arguments and repeated holes. It also explains why large arities can make profile enumeration expensive.

The source establishes two representation results. erase_step and erase_replay show that token-aware execution commutes with erasure to datum counts (source). Erasing a history is state-dependent: a duplicate datum admission becomes a supply, which leaves the erased regime unchanged. RegimeExt then identifies list representations with equal datum counts and form tables and membership-equal instance, rule and requirement collections; derivable_ext through validResponse_ext transport the stated observables (source). These are Lean-to-Lean theorems. Executable comparisons with Python and Rust have their own input populations; a general refinement theorem over either implementation's source has not been established.

7. What "minimal" must include

A response is an ordered finite list of NewDatum, NewForm, NewInst and NewRuleAll items. An instance can refer to a form added earlier in the list. A valid response applies successfully, separates its target, and leaves all previously imposed requirements separated. The declared costs are

$$\mu_1(\Delta) = |\Delta|, \qquad \mu_2(\Delta) = \left(|\Delta|, \sum_{\mathrm{NewForm}(a) \text{ in } \Delta} a \right),$$

with lexicographic comparison for μ_2. The sum counts new-form items, including different forms of equal arity.

The two challenge inputs are explicitly

$$\begin{aligned} R_L: &\quad D = \{0,1\}, \quad F = I = U = \emptyset, \quad Q = \{\{0,1\}\}, \\ R_F: &\quad D = \{0,1,2\}, \quad F = I = U = \emptyset, \quad Q = \{\{1,2\}\}. \end{aligned}$$

Datum 0 in R_F is the admitted initial token. The required pairs are stored as $(0,1)$ and $(1,2)$, respectively. In each input the target requirement is initially unsatisfied; a valid response must separate its endpoints.

Source result — exact first-separation cost. In the initial form-less, instance-less challenge regimes, the minimum μ_1 cost is exactly two in both readings, over the entire declared item grammar and every positive arity.

Proof. The empty response leaves the pair collapsed. A single NewDatum or NewForm leaves the instance set empty. A single NewInst or NewRuleAll cannot apply without an admitted form. Thus no response of at most one item separates the target. A fresh unary form followed by an instance at one demanded endpoint does separate it, and supplies the upper bound. The source formalizes these steps in cost1_insufficient_general, deltaU0_valid and deltaU0F_valid (source). □

Source result — ties and refined uniqueness. Reading L has infinitely many nonisomorphic minimum responses under μ_1. For each $k \geq 1$, add one k-ary form and its instance $(1, \ldots, 1)$. The all-hole context separates the two required data. Distinct arities cannot be identified by a form-arity

preserving isomorphism (deltaK_valid, markK_pairwise_noniso). The bounded arity-at-most-two census is a finite slice: L has six valid deltas in three classes; F has ten in five classes (source table).

Under μ_2, both readings have the unique minimum $(2, 1)$, up to the source's datum/form isomorphism, which preserves the unordered requirement. A successful two-item response with arity sum at most one must add a unary form and one instance. In L either endpoint choice is isomorphic to the other. In F a mark on the initial, unrequired datum separates neither demanded endpoint; the two successful endpoint choices are again isomorphic. These grammar-wide statements are mu2_unique and mu2_unique_F (L proof, F proof).

The cost belongs to this accounting language. Bundling a form and instance into one new primitive would change it. Two admitted items do not mean two observers, two certificates or two new data at every stage.

8. Closure and the next demand

Source result — finite round bound. Fix n data and a duplicate-free, well-formed initial requirement store Q_0. Suppose every round imposes a fresh canonically stored unordered pair and applies a response containing no NewDatum. Then a run of m successful rounds satisfies

$$2(|Q_0| + m) \leq n(n-1).$$

Proof. Responses leave the requirement store unchanged after imposition. Each fresh imposition therefore adds exactly one pair; the datum count stays fixed. The final store contains $|Q_0| + m$ distinct members of the $n(n-1)/2$ possible pairs. This is closedRun_round_bound; from an empty store it gives $2m \leq n(n-1)$ (closedRun_round_bound_from_empty, source). □

The declared closure policy chooses a currently collapsed pair and requires its separation while preserving all prior requirements. In any well-formed regime satisfying those requirements, a fresh unary form with one instance at an endpoint supplies such a response. Old-form derivability remains unchanged, so old witnesses survive (derivable_extend, mark_progress, source). Consequently an unguarded maximal run of this policy ends with all pairs separated: it cannot continue indefinitely by the bound, and it cannot stop on an unresolved pair for lack of a response. The companion theorem GenesisPaper.policy_terminal_separates formalizes this endpoint implication under well-formedness and preservation of existing requirements. A bounded implementation can instead stop at a search or resource limit; that is not the all-pairs-separated result.

Raw admission does not itself enforce this preservation policy. A unary instance at datum 0 separates data 0 and 1; adding the uniform rule for that form makes both profiles equal. More admitted structure can therefore mean fewer observational classes. The round bound tracks retained requirements, not a monotonicity theorem for arbitrary profile changes.

The closure policy supplies the demand to separate a collapsed pair. Its fixed-carrier endpoint does not by itself supply a fresh datum or a new goal. The later source continuation studied additional, explicitly declared policies and obtained further results:

- Both authorized continuation attempts closed and were merged. The second evaluated 144 engines from each of the two readings, giving 288 runs, with two-item and arity-two limits, `max_data = 4`, fifteen rounds, and a 250-node guard per run. All twelve rows of its FORCED census were decided under their stated stage or run readings (source census).

- For the cumulative element count $e(R) = |D| + |F| + |I| + |U| + |Q|$, a successful primitive step never decreases e, and a step changing the regime increases it by one. Switch events leave the regime unchanged. Thus no allowed history can return to a regime it has left, and every cycle has a constant regime. These are all-history results, including ereplay_elems_lt_of_ne, no_return and cycle_regime_const (source proof). They concern cumulative admitted elements, not datum count alone or physical irreversibility.
- The run-level two-ness property holds for all 64 responding LIVE engines, with its witness on every run path before a bound or guard. This does not assert two data at every stage. Other census claims have their own countermodels and bounds. In particular, one state reached after both five and six rounds shows that elapsed round count is not generally a function of the regime. Four engines carry the grammar forward with a stake and no stream within the measured bounds. Continuation beyond those bounds and a minimum number of declared inputs remain open (terminal account).

These results preserve both the finite constructive progress and its exact continuation obligations. They do not require reopening the frozen experiment.

9. An admitted witnessed difference

Once a relation R on a carrier A is explicitly admitted, suppose

$$\forall x \in A,\ \neg R(x,x), \qquad R(a,b).$$

Proposition 3 — endpoint distinction. Then a differs from b, so the support of this particular edge is the two-element set {a,b}.

Proof. If a=b, the supplied witness R(a,b) becomes R(a,a), contradicting irreflexivity. □

This argument does not force irreflexivity from opaque marks. It states the consequence of an admitted law and an inhabited relation. It says nothing about the total carrier size, the number of certificates witnessing the edge, or whether a reverse edge exists. A two-element carrier with only the edge a→b has one oriented relation occurrence and no b→a occurrence.

There is also a precise universal property for the chosen witness. Let D_2 have carrier $\{0,1\}$ and only the edge $0 \to 1$. Given $R(a,b)$, the function $f(0) = a, f(1) = b$ is the unique endpoint-preserving map from D_2 that preserves its edge. The proof checks the two inputs; the supplied witness verifies edge preservation. Under irreflexivity this map is injective by Proposition 3. The source formalizes D2_initial_pointed and irrefl_witness_ne (source). This represents the chosen witness inside the relation, not every edge of an arbitrary R, and does not require a reverse edge or a composition law.

10. From Genesis to the presentation questions

For a presentation S(n), its active demand register and its maps between stages, the practical questions now become precise:

1. Which data and operations are admitted at the current stage?
2. Which declared task cannot be answered with the current observations?
3. Which candidate extensions are permitted, and under which costs?
4. Which prior answers, inscriptions or algebraic laws must survive?

5. What evidence shows both preservation and resolution of the selected problem?

Two source results make the preservation questions concrete.

First, let equivalence relations $\sim_f$ and $\sim_c$ on the same set satisfy $x \sim_f y \Rightarrow x \sim_c y$. The map

$$q : X/\!\sim_f \longrightarrow X/\!\sim_c, \qquad [x]_f \longmapsto [x]_c$$

is well-defined and surjective. It is injective exactly when the reverse implication also holds. If an old identification fails in the finer relation, two distinct fine classes remain over one coarse class. **Proof:** use the same representative for surjectivity; equality of quotient classes gives both directions of the injectivity criterion. Successive such maps compose without choosing representatives. These are collapseMap_surjective, collapseMap_injective_iff_reverse_refinement and collapseMap_compose (source).

Second, for supplied maps $f : P \to Q$ and $g : Q \to P$, there exist injective maps $i : P \to C$, $j : Q \to C$ satisfying $i = j \circ f$ and $j = i \circ g$ if and only if

$$g \circ f = \mathrm{id}_P, \qquad f \circ g = \mathrm{id}_Q .$$

Proof. Substitute the two identifying equations and cancel the injective maps to get the round trips. Conversely take $C = P$, $i = \mathrm{id}$ and $j = g$; the round trips make g injective and give the equations. The source theorem is faithful_descent_iff_round_trip_coherent (source). Failure for this pair of maps requires retaining their presentation distinction; it does not rule out another translation.

Canonicalization has a related but separate contract. If a chosen representative $C(R)$ lies in the isomorphism orbit of R and is constant on each orbit, then $C(R) = C(S)$ exactly when $R \cong S$. An injective serialization of these representatives gives the same criterion for keys. The source establishes this abstract key_classification (source) and establishes relabelling preserves regime isomorphism (source). Connecting an optimized implementation to it still requires its actual normalization, serialization and search hypotheses. List order and duplicate conventions cannot be discarded merely because the abstract theorem holds.

The companion GenesisPaper.serializedRelabel_sound in lean4/PaperCanonical.lean formalizes a concrete soundness direction: if two well-formed regimes have equal sorted five-field tuples after datum and form relabeling by supplied bijections, they are isomorphic. Its proof returns the composed isomorphism and establishes that collection sorting preserves membership. The companion GenesisPaper.Wire.portableBytes_injective in lean4/PaperWire.lean formalizes the length-prefixed field layout and its unsigned 64-bit big-endian encoding. Every encoded integer, including a collection length, must be less than 2^{64}. A structural decoder recovers the tuple from its words, and the fixed-width byte encoding retains those words. Thus equal encoded relabellings imply regime isomorphism under the stated well-formedness, range and supplied-bijection hypotheses (portableBytes_sound).

The companions lean4/PaperOrbit.lean and lean4/PaperSearch.lean close the concrete exhaustive-search classification question. For well-formed regimes with duplicate-free instance, rule and requirement collections, an isomorphism constructs bounded datum and form bijections and preserves the entire set of serialized relabellings. An insertion-based enumeration is proved to contain exactly all permutations of each finite index range. The executable minimum search attains a

least structural tuple (canonicalKey_attained), and its keys are equal exactly when the input regimes are isomorphic (canonicalKey_classification). Composing this with byte injectivity gives canonicalBytes_classification under the unsigned-word bounds. These theorems supply the enumeration and minimum-selection arguments; they do not assume orbit invariance or an orbit-choice oracle. General refinement of the actual Python and Rust source programs, including their optimized searches, remains a separate engineering obligation.

These criteria permit a mathematical representation to be checked against its claimed source structure. Selecting a new operation remains a declared construction decision, with its preservation obligations stated.

11. Engineering follow-up

There is now a concrete software connection: `omega_wdp::genesis_construction` calls the canonical omega_ooutwire_primitives admission implementation and compares every accepted prefix with WDP's deduplicating execution model, including all five regime fields. Starting from a literal supply of `<>`, its explicit incidence interpretation reads admitted instances as simplices, checks vertices and faces, and delegates exact rational homology to the existing mathematics code. The triangle boundary has Betti numbers (1, 1); admitting its face gives (1, 0, 0). The construction account states the interpretation and resource limits. This is an implemented walk with checked executions, not a general Lean refinement theorem over Rust or a claim that the incidence interpretation is selected by the token alone.

PFP is the separate engineering follow-up: candidate, checker, receipt and authorized use must be bound to their task and history. Its intended consolidation draws on the existing `011-NNN` source range. OEWOLAND, OEWAS, OEEIS and OPCIV have no publication commitment here. No implementation or disclosure of them is required to make the mathematical statements above intelligible.

Mechanism inventory

Mechanism	Mathematical evidence	Executable scope
Observation and history report descent	Carrier-general companion proofs and pinned finite compression proofs	Finite observation profiles in Python and Rust; no inference about trial success rates
Admission, erasure and collection extensionality	Pinned Lean transition and representation theorems	Frozen Python model and Rust companion; the documented name collision distinguishes them
Minimum responses and closure	Grammar-wide cost, nonuniqueness, uniqueness, preservation and round-bound proofs	Arity-at-most-two minimum census; explicit finite preservation population
Chosen witnessed edge and presentation retention	Pinned pointed-graph, quotient-retention and round-trip theorems	These are mathematical structure results; no forced physical interpretation
Canonical classification	Concrete exhaustive-search and byte-key classification for well-formed set regimes	Versioned portable protocol and native Lean/Python/Rust agreement; general source-code refinement remains open
Continuation	Pinned all-history growth and no-return proofs	Historical census bounds remain those of the cited source
Primitive-to-incidence interpretation	Explicit construction choice	The previously reported WDP triangle executions remain separate from this standalone primitive reproduction

12. Sources, provenance and verification status

The source manifest records the historical inputs. The mechanism inventory above distinguishes formal results, computations and construction choices. The complete source coverage record inventories 3,147 tracked files and assigns all 121 research directories a disposition. Inventory coverage is not an independent verification of every proof in those files.

The final source closeout records the author's September 28 freeze of snapshot 90ab34a056d80bb104 8d926abcfdfef81e578fe1, approved with merge 50d15a44. Frozen reporting files retain their earlier "Not frozen" headers; the later approval governs their status. Three independent verdicts stand:

Axis	Final source status
Research phase	CLOSED_WITH_OPEN_OBSTRUCTIONS
Alpha reference implementation	READY_WITHIN_DECLARED_SCOPE
Original seed completion gate	UNSATISFIED

The completion gate still records unmet live-injection and comparison clauses, and an unestablished

input-count floor. Companion work does not retrospectively rewrite that frozen verdict. It can establish additional, separately identified results.

A source implementation defect also remains part of the record. Its delta_events function generates a supposedly fresh label `w<n>` from the current datum count. If the sole admitted token is already w1, a NewDatum item emits `AdmitDatum("w1")`; deduplication leaves the state unchanged (source). This was reproduced during this review. Correcting the Rust implementation is companion work; comparisons must identify both their collision-free agreement scope and cases where the corrected behavior differs.

The standalone companion includes unchanged pinned Python and Lean proof sources, a self-contained Rust crate, and executable reproduction commands. Lean 4.33.0 compiles the pinned root, parked modules, canonical-minimizer research modules, second-run kernel and continuation proofs, and the companion manuscript corollaries. The compiled census reports 2,475 non-internal theorem constants, including compiler-generated theorems. Pinned modules use only propext and Quot.sound; the carrier-general image-factorization companion also uses Classical.choice. This population is neither a count of manuscript claims nor verification of all 121 source research directories.

The standalone replay reproduces the four minimum-response counts in §7, checks the context formula in 35 cases, and verifies fresh-mark preservation for 3,996 targets across 1,436 regimes. Native Rust and frozen Python agree on every context observation and portable key in that regime population. The native proved Lean minimum search agrees with their portable keys on those 1,436 labelled regimes; two additional cases check mixed arities and unsorted list representations. Its interface rejects 17 malformed or over-budget inputs. The Rust primitive suite has 135 passing tests; 55 unrelated projection-study tests are excluded from that result. No new model trial or full frozen-source completion-gate run is claimed. Commands and scope are in the README.

Python's historical refined keys sort colors by repr; Rust's local keys use structural tuple ordering. They can select different labelled representatives of the same orbit. These local keys and their derived fingerprints must not be exchanged as identical values. The companion provides portable_key_v1 in both languages: minimize the structural serialization over all bijection pairs, then use the versioned, length-prefixed unsigned-integer encoding in the protocol specification. Exhausting its declared permutation budget returns an error. Existing local keys retain their original conventions. The finite agreement above validates the tested implementations; it does not assert a compiler-verified refinement proof for all source code. Both decoders also agree on 72 explicit boundary and malformed-input cases. The compiled Lean wire encoder agrees with Python on 1,439 tuples: the 1,436 regime keys and three additional boundary vectors. These comparisons include maximum unsigned 64-bit words. Decoder acceptance checks the tuple's shape, well-formedness and collection order; it does not certify an orbit minimum.

The source reader includes the cited education chapters, three trial transcripts, historical mathematical accounts and closeout record. The original repositories are private; the manuscript's citations point to bundled reading copies or executable sources. Their manifest records original paths, revisions, Git blob identities and SHA-256 hashes. The proof and Python sources match the exact Git bytes at f1d56c01; later companion changes do not silently change their referents. Repository access is unnecessary for following these citations.